

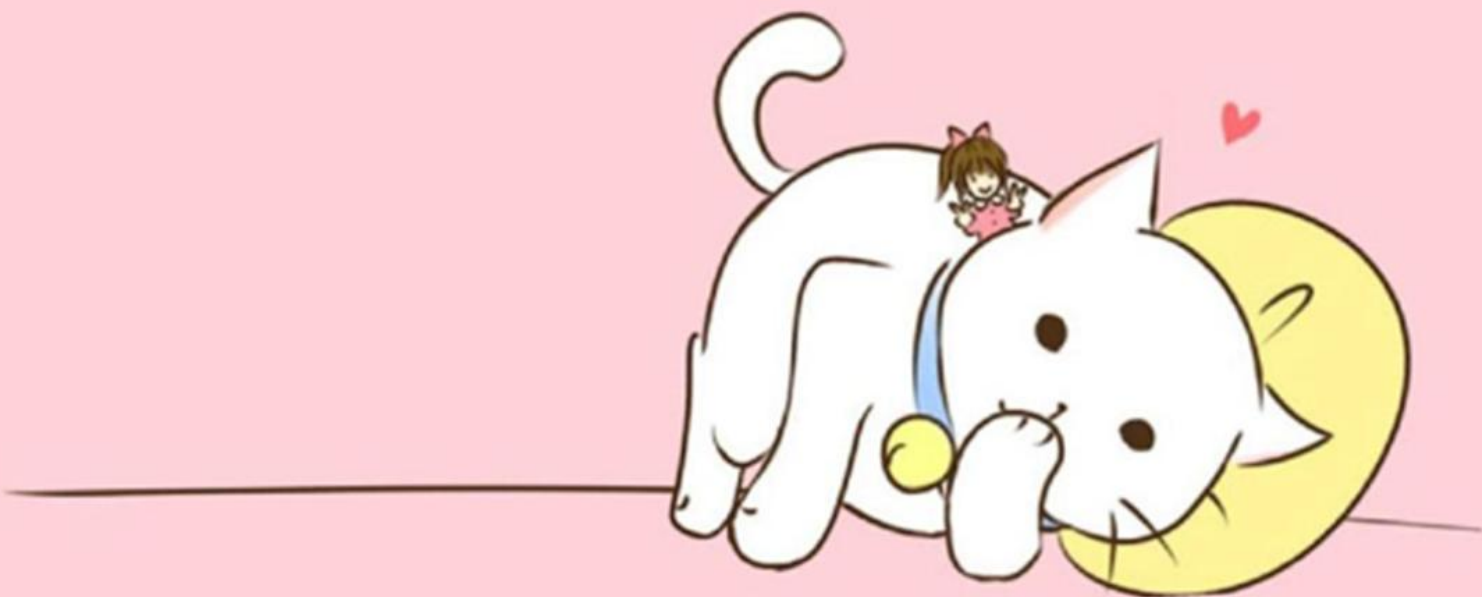
N诺考研系列



2023 版

计算机考研机试攻略

满分篇



公众号：N诺考研

官网：www.noobdream.com

计算机考研保研必备神器

全国300所院校历年真题，尽在N诺

N 诺考研系列

计算机考研机试攻略

满分篇



N 诺课程教研团队

2022. 05. 20 更新

写在前面的话

相信各位同学都知道，N 诺是一个大佬云集的平台。N 诺每周都会定期举办比赛，让同学们在日复一日的枯燥生活中找到一种乐趣。

本书是 N 诺专为计算机考研的同学精心准备的一本**神书**，为什么说它是一本神书，它到底神在什么地方？

- 1、这本书是 N 诺邀请众多 **CSP 大佬**、**ACM 大佬**、**BAT 专业大佬**以及往年**机试高分大佬**共同修订而成。
- 2、这本书与市面上的书都不一样，这是一本专门针对**计算机考研机试**而精心编写而成的书籍。
- 3、这本书上的**例题讲解**以及**习题**都可以在 N 诺上找到进行练习，并且每道题都有大佬们发布的**题解**可以学习。
- 4、N 诺有自己的官方群，群里大佬遍地走，遇到问题在群里可以随时提问。群里**神仙**很多，不怕没有问题，就怕问题太简单。

相信同学们一定听过各种各样的 OJ 平台，但是那些平台不是为了计算机考研而准备的，而 N 诺是**唯一**一个纯粹为计算机考研而准备的学习平台。

相信每一个同学在学习本书之前，都觉得机试是一个很头疼的问题，机试有可能**速成**吗？我可以一周就**变得很强**吗？我们可以在短短两三周之内的学习就能拿到**机试高分**吗？

别人可不可以我不知道，但是在 N 诺这里，你就可以。

虽然考研机试题目千千万，但是你可以“一招鲜，吃遍天”。

管它乱七八糟的排序，我就一个 sort 你能拿我怎么办？

管它查来查去的问题，我会 map 我怕谁？

管它飘来飘去的规律，我有 OEIS 神器坐着看你秀！

管它眼观缭乱的算法，我有 N 诺万能算法模板怕过谁？

本书虽然不能让你变成一个算法高手，但是本书可以让你快速变成一个机试高手，这也是本书最重要的特点，以解决同学们的实际需求为目的。你可以不会算法，你可以不必弄懂算法的原理，你只要学会本书教给你的各种技巧，就足以应对 99.9% 的情况。

N 诺的课程教研团队全是由大佬组成，每个人都做过几千道编程题目，做编程题的经验十分丰富，我们分析近百所院校的历年机试真题，从中发现了各个学校的机试真题都有相同的规律，万变不离其中，于是本书也就诞生了。

计算机考研机试攻略交流群请扫描下方二维码或直接搜索群号：960036920



群名称：N诺 - 计算机机试交流群
群号：960036920

本书配套精讲视频：<https://www.bilibili.com/video/av91373687>

关于 N 诺

N 诺是全国最大的计算机考研在线学习平台。N 诺致力于为同学们提供一个良好的网络学习环境，一个与大佬们随时随地交流的舒适空间。如果你不知道 N 诺，那么你已经输在起跑线上了，因为 N 诺是 - 计算机学习考研必备神器。

N 诺整理了**计算机考研报考指南**，帮助你了解考研的点点滴滴赢在起跑线上。

<http://www.noobdream.com/media/upload/2020/01/29/guide.pdf>

在 N 诺，你可以查询各个院校的**考研信息**，包括分数线、录取人数、考试大纲、导师信息、经验交流等信息，应有尽有。

<http://www.noobdream.com/schoollist/>

在 N 诺，你可以尽情的刷各个科目的**题库**，还可以将题目加入**错题本**方便以后复习，也可以写下自己的**学习笔记**，记录考研过程中跌宕起伏。

<http://www.noobdream.com/Practice/index/>

在 N 诺，你可以在**讨论区**里发表你的问题或感想，与全国几百万考研 er 分享你的喜怒哀乐。

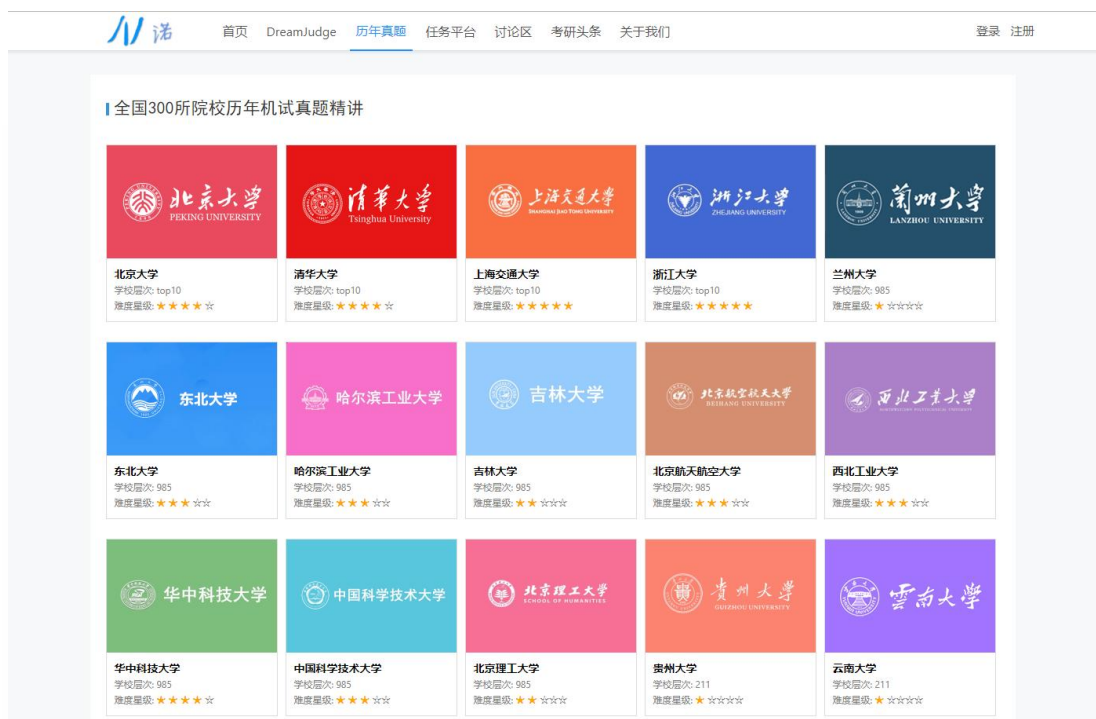
<http://www.noobdream.com/forum/0/>

在 N 诺，你可以将你不用**的书籍或资料**放到**二手交易市场**，既能帮助他人，还能为自己省下一笔当初买书买资料的费用。

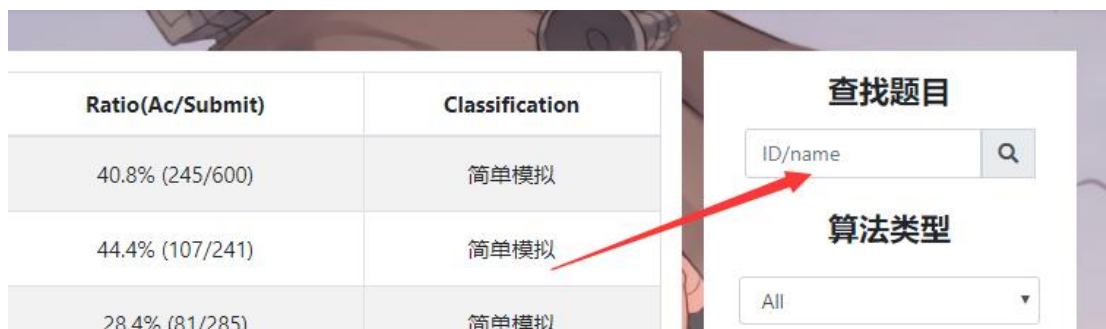
<http://www.noobdream.com/Task/tasklist/>

如何使用本书？

访问 N 诺平台（www.noobdream.com）的历年真题，即可查询到全国各个学校的历年机试真题。



如下图输入课后习题的编号即可搜索到题目



小技巧：N 诺是可以随意更换皮肤的哟，在右上角头像旁边。

目录

写在前面的话	2
关于 N 诺	4
如何使用本书?	5
目录	6
第一章 技巧之巅	7
1.1 输入输出加速外挂	8
1.2 调试技巧	10
1.3 位运算技巧	12
1.4 考试最佳策略	16
1.5 预处理与打表技巧	18
1.6 对数器技巧	21
第二章 满分之路上	24
2.1 计算几何基础	25
2.2 进阶背包问题	32
2.3 毛毛虫算法	36
2.4 博弈类问题	40
2.5 二分答案技巧	50
2.6 前缀和技巧	53
2.7 双指针技巧	56
第三章 满分之路中	58
3.1 线段树单点更新	59
3.2 线段树区间更新	65
3.3 字符串匹配问题	72
3.4 二分图的匹配问题	79
3.5 kmp 算法的应用	83
3.6 路径进阶问题	84
3.7 图的连通性问题	91
3.8 单调队列/栈	95
第四章 满分之路下	98
4.1 容斥与抽屉原理	99
4.2 除法取模问题	103
4.3 组合数取模类问题	105
4.4 矩阵快速幂	109
4.5 带状态压缩的搜索	112
4.6 数位类型动态规划	116
4.7 FFT 快速傅里叶变换	120
4.8 机试押题	125
完结撒花	126
N 诺考研系列图书	127
计算机考研忠告	错误! 未定义书签。

第一章 技巧之巅

建议大家在阅读本书之前先阅读本书的姊妹篇：**计算机考研机试攻略 - 高分篇**。

这本书是高分篇的加强版，适合学习完高分篇的同学继续学习，拿到机试满分。

本章是 N 诺计算机考研机试攻略满分篇的第一章，本章我们带领大家领略技巧的极致魅力。本章包括：输入输出加速外挂、调试技巧、位运算技巧、考试最佳策略、预处理与打表技巧、对数器技巧等内容。可以帮助读者迅速掌握一些使用的考试技巧，完成更高难度的挑战。



本书配套视频精讲：<https://www.bilibili.com/video/av91373687>

1.1 输入输出加速外挂

有的时候题目的输入数据量比较大, 比如要输入 10W 和数字进行排序。这个时候, 如果你直接使用 C++ 的 `cin` 和 `cout` 函数进行输入输出, 有很大的概率会超出题目的时间限制。

在这种情况下, 需要的优化的就不再是你的算法过程, 而是你的读写数据的速度优化。如果使用 `cin` 和 `cout` 函数进行输入输出, 我们建议你在 `main()` 里首先写入下面两行代码。

C++ `cin/cout` 加速

```
1. ios_base::sync_with_stdio(0);  
2. cin.tie(0); cout.tie(0);
```

如果题目的输入量巨大, 比如要输入 100W 个数字, 这个时候我们建议你使用 C 语言的 `scanf` 和 `printf` 语句进行输入输出。

如果在这种情况下还超时, 那么你就需要用到下面的输入输出加速外挂了。

```
1. //适用于正负整数  
2. template <class T>  
3. inline bool scan_d(T &ret) {  
4.     char c; int sgn;  
5.     if(c=getchar(),c==EOF) return 0; //EOF  
6.     while(c!='-'&&(c<'0' || c>'9')) c=getchar();  
7.     sgn=(c=='-')?-1:1;  
8.     ret=(c=='-')?0:(c-'0');  
9.     while(c=getchar(),c>='0'&&c<='9') ret=ret*10+(c-'0');  
10.    ret*=sgn;  
11.    return 1;  
12. }  
13. inline void out(int x) {  
14.     if(x>9) out(x/10);  
15.     putchar(x%10+'0');  
16. }
```

加速外挂原理:

getchar 的速度 快于 scanf 的速度

速度比较

getchar > scanf > cin

putchar > printf > cout

参考代码

```
1. // 求 1 + n 的和
2. #include <bits/stdc++.h>
3. using namespace std;
4.
5. //适用于正负整数
6. template <class T>
7. inline bool scan_d(T &ret) {
8.     char c; int sgn;
9.     if(c=getchar(),c==EOF) return 0; //EOF
10.    while(c!='-'&&(c<'0' || c>'9')) c=getchar();
11.    sgn=(c=='-')?-1:1;
12.    ret=(c=='-')?0:(c-'0');
13.    while(c=getchar(),c>='0'&&c<='9') ret=ret*10+(c-'0');
14.    ret*=sgn;
15.    return 1;
16. }
17. inline void out(int x) {
18.     if(x>9) out(x/10);
19.     putchar(x%10+'0');
20. }
21. // 请注意只有在大量输入或大量输出的时候才能看出时间的区别
22. int main() {
23.     int n;
24.     scan_d(n); //加速输入
25.     long long sum = 0;
26.     for (int i = 1; i <= n; i++)
27.         sum += i;
28.     out(sum); //加速输出
29.     return 0;
30. }
```

1.2 调试技巧

调试是我们在编写程序时不得不经历的过程, 我们这一节来讲解如何快速定位错误的调试技巧。

首先, 编译错误不在我们的讨论范围之内, 语法上的错误问题相信学过高分篇的同学都不会再问了。

其实, 断点调试也不在我们的讨论范围之内。

这里讲一下我们为什么不建议大家使用断点调试, 断点调试基本上是每个开发者都会的调试技巧。

但是我们的机试有一些特殊的情况。

- 1、机试的代码往往很短, 几行到几十行不等。
- 2、比赛中争分夺秒, 我们对调试时间要求更为迫切。
- 3、我们的错误往往是由于代码细节没考虑周全导致的。

所以, 断点调试更适于项目代码中且对时间的迫切度没有那么高的情况。

接下来, 给大家介绍一种超级棒的调试技巧, 当你熟练掌握它以后你会深深的迷恋上它。

输出调试

顾名思义，就是**通过输出的方式定位**我们的错误所在。

大部分没有足够调试经验的同学使用输出调试的时候，不知道应该如何使用输出调试。

如果从前往后逐条语句输出调试去排查错误，那么很容易要找很久。

反之从后往前逐条语句输出调试去排查错误，那么也很容易要找很久。

不知道同学们是不是发现了点什么，对，没错，上面两种方法是不是就是顺序查找的方法。

那么，与此对应的就应该是二分查找的方法。

使用**二分查找**的思想来调试定位错误，可以更快，更节约时间。

特别注意：输出调试完成之后提交代码之前一定要删除或注释掉调试信息。

1.3 位运算技巧

位运算，相信所有的同学都有所了解。

那么关于位运算有哪些特殊的技巧呢？

速度比较

取模时间 > 四则运算时间 > 位运算时间

所以对于一个语句

```
1. if (a % 2 == 1) {  
2.     a /= 2;  
3. }
```

我们可以优化成为

```
1. if (a & 1 == 1) {  
2.     a >>= 1;  
3. }
```

异或运算的特殊性

异或同一个数 2 次或者偶数次，那么本身的价值不变。

例如：

$$a \oplus b \oplus b = a$$

$$x \oplus y \oplus y \oplus y \oplus y = x$$

接下来我们来看一下它的应用。

缺页问题

题目描述：

小明有一本很喜欢的漫画书，但是由于他上课的时候偷看这本漫画书被老师发现，于是老师将

小明的漫画书撕了扔进垃圾桶。

小明放学后从垃圾桶里将漫画书捡了回来，却发现漫画书少了一页。小明的漫画书一共有 N 页，由于现在页码顺序错乱，小明也不知道是少了哪一页，聪明的你能告诉小明缺失的是哪一页吗？

输入描述：

第一行输入一个整数 N ($N < 1000000$)，表示小明书的总页数。

第二行输入 $N-1$ 个数整数，代表小明找到的页码。

输出描述：

请输出小明缺失的页码在单独的一行。

请注意：本题的空间很小，要求你尽量不使用额外的空间来解决。

输入样例#：

6

6 2 1 5 3

输出样例#：

4

题目来源：

DreamJudge 1506

题目解析：由于本题要求我们以尽量小的空间来解决问题，所以我们不能够使用数组来存储每一个数。那么我们应该怎么办呢？这个时候可以想到异或运算的特殊技巧，同一个数异或两次那么就会消除，如果我们提前将 1 到 N 的所有数字进行异或处理，然后再去异或输入的 $N-1$ 个数，那么答案就是缺失的那个数。

参考代码

```
1. #include<bits/stdc++.h>
2. using namespace std;
3.
4. int main(){
```



```
5.     int n, x;
6.     scanf("%d", &n);
7.     int sum = 0;
8.     for (int i = 1; i <= n; i++) {
9.         sum ^= i;
10.    }
11.    for (int i = 1; i < n; i++) {
12.        scanf("%d", &x);
13.        sum ^= x;
14.    }
15.    printf("%d\n", sum);
16.    return 0;
17. }
```

常见位运算问题

1. 位操作实现乘除法

数 a 向右移一位, 相当于将 a 除以 2; 数 a 向左移一位, 相当于将 a 乘以 2

```
1.  int a = 2;
2.  a >> 1; ---> 1
3.  a << 1; ---> 4
```

2. 取相反数

思路就是取反并加 1, 也即 $\sim n + 1$ 或者 $(n \wedge -1) + 1$ 。

3. 判断奇偶性

```
1.  /* 判断是否是奇数 */
2.  bool is_odd(int n)
3.  {
4.      return (n & 1 == 1);
5.  }
```

4. 不用临时变量交换两个数

```
1. a ^= b;  
2. b ^= a; // 相当于 b = b ^ ( a ^ b );  
3. a ^= b;
```

5. 统计二进制中 1 的个数

```
1. count = 0  
2. while(a){  
3.     a = a & (a - 1);  
4.     count++;  
5. }
```

题目练习

DreamJudge 1118 将军的书

noobdream.com

1.4 考试最佳策略

很多上机考试经验不够丰富的同学，往往很难发挥出自己应有的水平。

下面我们来讲一下如何应对一场考研的上机考试。

提前准备

- 1、算法模板一定要提前准备好，不管是单独打印还是记录在书上。
- 2、算法模板一定要验证它的正确性，在验证的过程中也知道了该如何使用。
- 3、手机调成静音模式，最好再开一个飞行模式，不要上交以备不时之需。

开考前

- 1、验证鼠标键盘是否可用，如果不可用及时向老师反应，更换电脑。
- 2、验证 IDE（如：codeblocks）是否可用，如果不可用及时向老师反应，更换电脑。
- 3、提前将头文件和主函数框架写好，最好创建两个文件，代码复制一份就行。因为在做题过程中遇到卡题情况，可以及时切换到另一道题上继续写。

考试中

- 1、机试中的题目难度不是从简单到难，难度是随机的，所以刚开始一定要将所有题目都看一遍。
- 2、找到你认为最简的那道题开始做，记住，一定要从最简单的题目开始做。
- 3、考试过程中要注意看排行榜，通过人数最多的题目一般都是最简单的题目。
- 4、注意：要看通过人数的多少来判断难易程度，而不是第一个人通过的时间来判断。
- 5、如果你的水平强，可以选择先做那种代码不长但是需要算法思维的题，快速解出来将排行榜带偏，给其他人一种这题最简单的感觉，然后让他们死磕这题，成功让竞争对手翻车。
- 6、之所以要赤裸裸的将上一条写出来，就是为了告诫那些头铁的同学，不要在一棵树上吊死，都走复试这一步了，还这么死脑筋基本没得救了，
- 7、考试的时候遇到规律题，千万不要忘记 OEIS 这个神器，特别是周围上厕所的同学开始变多。
- 8、当无法通过一个题的时候，先看看有没有其他能做的题，如果也没有其他题能做了。

这个时候你就可以使用很多特殊的办法，首先，你一定要相信，机试的数据一定不够强。一般情况下，机试的判题数据都会找学生帮忙生成，往往强度就不会很高，这个要看学生的尽责程度。如果是老师自己构造的数据，你睡着都会笑醒。

9、不要被题目的数据范围吓到，有可能后台都是小数据，没有更好的解法的时候一定要试试暴力。

10、我们往往被卡都是因为算法不够优秀导致超时，一方面我们可以强行优化输入输出加速来看看能不能水过去。另一方面可以采用小数据暴力，大数据随机的思想来解决问题。

小数据暴力、大数据随机

原理：由于大数据出多组容易导致判题很慢，所以往往不会有很多组。另外对于特殊的数据可能需要手动构造，大数据构造起来麻烦，还要自己构思生成数据的代码。所以一般都是用小数据来验证算法正确性，再加上两组大数据组验证算法复杂度。

举例说明

背包问题

我们背包问题一般使用动态规划来解，但是你不会怎么办？

那么我们就可以小数据暴力搜索，大数据直接贪心。对于数据不够强的题就能水过去。

更简单的例子

给你很多个数，要你从中找出最大的数。

你需要从第一个找到最后一个才能判断出最大的数是哪个吗？

如果是这样几个数：5 2 7 1 6 3

我们是不是只需要找到一半就能找到最大那个数了

如果是这样几个数：8 2 7 1 6 3

我们是不是只需要找到第一个就能找到最大那个数了

一般情况下，判题数据就像上面那样，你可以通过不严谨的代码通过题目。

主要是领悟思想，不同的题都有不同策略。

1.5 预处理与打表技巧

预处理

预处理是指将答案提前处理好然后再进行查询的方法。

什么时候会用到预处理？

查询量特别大的时候。

例题

我们要查询斐波那契数列的第 n 项, n ($n < 10000$)。

并且我们要查询 10 万次。

错误的超时解法



```
1. #include<bits/stdc++.h>
2. using namespace std;
3.
4. int f[10005];
5. int main(){
6.     int t;
7.     scanf("%d", &t); //查询次数
8.     while (t--) {
9.         int n;
10.        scanf("%d", &n); //查询第 k 项
11.        f[1] = 1;
12.        f[2] = 1;
13.        for (int i = 3; i <= n; i++) {
14.            f[i] = f[i-1] + f[i-2];
15.        }
16.        printf("%d\n", f[n]);
17.    }
18.    return 0;
19. }
```

超时分析：每一次查询，我们都递推了一遍斐波那契数列，如果每一次查询的都是最后一项。那么最坏情况就是：100000*10000，必然是会超时的。

正确的预处理解法

```
1. #include<bits/stdc++.h>
2. using namespace std;
3.
4. int f[10005];
5. int main(){
6.     f[1] = 1;
7.     f[2] = 1;
8.     for (int i = 3; i <= 10000; i++) {
9.         f[i] = f[i-1] + f[i-2];
10.    }
11.    int t;
12.    scanf("%d", &t); // 查询次数
13.    while (t--) {
14.        int n;
15.        scanf("%d", &n); // 查询第 k 项
16.        printf("%d\n", f[n]);
17.    }
18.    return 0;
19. }
```

可以发现两段代码惊人的相似，有什么区别呢？

区别在于下面的代码只会递推一遍斐波那契数列，然后将所有的答案都存储于 f 数组中。

一般打表技巧

打表是指我们提前使用暴力的方法, 将答案记录下来写到纸上或代码中, 然后再根据题目的实际需求去直接输出提前记录好的答案。

例题

求一个数 x 的 100000000 次方模 2333 的值是多少。

x 的范围是 ($1 \leq x \leq 10$)

解析: 这个时候我们如果不会二分快速幂的话, 可以直接用一个暴力的代码将 1 到 10 的答案分别求出来, 可能要等上几分钟, 然后直接判断输入的值对应输出结果即可。

打表找规律

在一些可能存在规律的问题中, 我们可以通过暴力打表求出前几十项的值, 发现其中的规律, 从而帮助我们解出该题。

例题

输入一个整数 a ($a < 10^9$), b 可以取任意值, 求 $a \% b$ 的可能产生多少种不同的结果。

解析: 乍一看, 我们一脸懵逼。这个时候你只要去打表分析, 首先 b 大于 a 就重复了, 那么我们只需要对于任意一个 a , 去枚举 b 的值, 看 $a \% b$ 会产生多少种不同的结果即可。通过打表分析, 你会发现: 当 $a = 1$ 时, 结果 $ans = 1$, 当 a 为偶数时, $ans = a / 2 + 1$, 当 a 为奇数时, $ans = a / 2 + 2$ 。

题目练习

DreamJudge 1488 数字填充

1.6 对数器技巧

大多数同学应该都有过这样的经历...

这道题我做了半天怎么还是错的啊, 看了别人正确的代码跟我的差不多啊, 我到底错在哪里了啊? 抓狂...

然后开始疯狂 diss 管理员, 管理员在吗? 你这个小破站, 为什么不能告诉我是哪一组数据出错了? 你有没有考虑过我们这种新手的感受? 你看看别人某站, 错误还会给我反馈错误的数
据, 你们是不是技术不行啊?

管理员的无奈: 大部分题目的极限数据都是上千上万行, 反馈给你也没用的, 你要花大量的时间去查找去比较, 还不如再认真检查一下代码。

但是, 当你学会这一节的内容之后, 这一切的烦恼都不存在了。

使用对数器进行对拍, 可以快速的找到的你的代码错误所在。

简单说, 就是将你的代码和正确的代码放在一起, 然后随机生成一些符合题目的要求的数据, 然后比较你的代码的输出结果和正确代码的输出结果是否一致, 从而来验证你的代码的正确性。

当然, 随机的数据越多, 正确性也就越高。

下面我们举个排序的例子

我自己写了一个排序的算法, 但是我不知道我写的这个排序算法到底有没有问题, 但是我拿到别人的正确的排序的算法代码, 我如何比较呢?

我的排序算法代码

```
1. for (int i = 1; i <= n; i++)  
2.     for (int j = 1; j < n; j++)  
3.         if (b[j] > b[j+1])  
4.             swap(b[j], b[j+1]);
```

正确的排序算法代码

```
1. sort(c+1, c+n+1);
```

下面就是完整的对数器使用的模板

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. struct node {
5.     int a[105]; //随机的数
6.     int b[105]; //我的方法排序后的数
7.     int c[105]; //正确的方法排序后的数
8.     int n;
9.     // 按照题意随机一些输入数据
10. void Rand() {
11.     n = rand()%100 + 1;
12.     for (int i = 1; i <= n; i++)
13.         a[i] = rand()%10000;
14. }
15. // 使用我的解法得到的答案
16. void My_method() {
17.     for (int i = 1; i <= n; i++)
18.         b[i] = a[i];
19.     for (int i = 1; i <= n; i++)
20.         for (int j = 1; j < n; j++)
21.             if (b[j] > b[j+1])
22.                 swap(b[j], b[j+1]);
23. }
24. // 使用其他正确的解法得到的答案
25. void Right_method() {
26.     for (int i = 1; i <= n; i++)
27.         c[i] = a[i];
28.     sort(c+1, c+n+1);
29. }
30. // 比较我的答案和正确答案是否一致
31. // 如果不一致输出测试数据然后分析
32. int Check() {
33.     for (int i = 1; i <= n; i++) {
34.         if (b[i] != c[i]) {
35.             printf("my code is error!\n");
36.             printf("the test data:\n");
37.             for (int j = 1; j <= n; j++) {
38.                 printf("%d ", a[j]);
39.             }
40.             printf("\n");
```

```
41.         return 1;
42.     }
43. }
44.     return 0;
45. }
46. };
47.
48.
49. int main() {
50.     srand(time(NULL)); //将时间作为随机种子
51.     int t = 100; //样本大小
52.     while (t--) {
53.         node code;
54.         code.Rand();
55.         code.My_method();
56.         code.Right_method();
57.         if (code.Check()) return 0;
58.     }
59.     printf("my code is right!\n");
60. }
```

代码分析：首先随机 100 组输入数据，然后使用自己的算法去跑出结果，然后使用正确的算法去跑出结果，最后比较两个结果是否一致，如果不一致就输出是哪一组数据导致的结果不一致，最后分析自己的算法问题所在。

第二章 满分之路上

高分篇的内容已经将机试可能涉及到的题型及考点都一一列举出来, 并给大家提供了练习题目, 帮助大家巩固基础加深理解。在大多数院校的机试中, 学会高分篇的内容已经足够考到 90 分以上的成绩, 题目不难的情况下甚至能拿到 100 分。在少数难度较大的院校也足够拿到 80 分左右的分数, 如果你基础比较薄弱, 我们建议你掌握高分篇的内容就可以了。本书满分之路的内容不是常考点, 只是有一定可能会考到, 我们建议基础好的同学继续学习增加拿满分的把握。

距离满分只有一步之遥, 你! 愿意放弃吗?

本章我们重点讲解一些较为深入的题型, 包括计算几何基础、进阶背包问题、毛毛虫算法、博弈类问题、二分答案技巧和前缀和技巧等内容。希望能帮助读者更好的掌握计算机考研机试中所涉及到的各类较难的问题。

提示: 满分篇的内容选择性的学就行, 不要求全部掌握, 会其中一部分就很厉害了。

本书配套视频精讲: <https://www.bilibili.com/video/av91373687>

2.1 计算几何基础

在考研机试中, 有的时候会考到一些基础的计算几何知识, 其中包括点积、叉积、凸包等内容。

首先在二维坐标下介绍一些定义:

点: $A(x_1, y_1)$, $B(x_2, y_2)$

向量: 向量 $AB = (x_2 - x_1, y_2 - y_1) = (x, y)$

向量的模 $|AB| = \sqrt{x^2 + y^2}$

点积

向量的点积: 结果为 $x_1 * x_2 + y_1 * y_2$ 。

点积的结果是一个数值。

点积的集合意义: 我们以向量 a 向向量 b 做垂线, 则 $|a| * \cos(a, b)$ 为 a 在向量 b 上的投影, 即点积是一个向量在另一个向量上的投影乘以另一个向量, 且满足交换律。

应用: 可以根据集合意义求两向量的夹角, $\cos(a, b) = (\text{向量 } a * \text{向量 } b) / (|a| * |b|) = x_1 * x_2 + y_1 * y_2 / (|a| * |b|)$

noobdream.com

叉积

向量的叉积: 结果为 $x_1 * y_2 - x_2 * y_1$

叉积的结果也是一个向量, 是垂直于向量 a, b 所形成的平面, 如果看成三维坐标的话是在 z 轴上, 上面结果是它的模。

方向判定: 右手定则, (右手半握, 大拇指垂直向上, 四指右向量 a 握向 b , 大拇指的方向就是叉积的方向)

叉积的集合意义:

- 1、其结果是 a 和 b 为相邻边形成平行四边形的面积。
- 2、结果有正有负, 有 $\sin(a, b)$ 可知和其夹角有关, 夹角大于 180° 为负值。
- 3、叉积不满足交换律

应用：

1、通过结果的正负判断两矢量之间的顺逆时针关系

若 $a \times b > 0$ 表示 a 在 b 的顺时针方向上

若 $a \times b < 0$ 表示 a 在 b 的逆时针方向上

若 $a \times b = 0$ 表示 a 在 b 共线，但不确定方向是否相同

2、判断折线拐向，可转化为判断第三点在前两的形成直线的顺逆时针方向，然后判断拐向。

3、判断一个点在一条直线的那一侧，同样上面的方法。

4、判断点是否在线段上，可利用叉乘首先判断是否共线，然后在判断是否在其上。

5、判断两条直线是否想交（跨立实验）

根据判断点在直线那一侧我们可以判断一个线段的上两点分别在另一个线段的两侧，当然这是不够的，因为我们画图发现这样只能够让直线想交，而不是线段，所以我们还要对另一条线段也进行相同的判断就 ok。

凸包

凸多边形

凸多边形是指所有内角大小都在 $[0, \pi]$ 范围内的 简单多边形 。

凸包

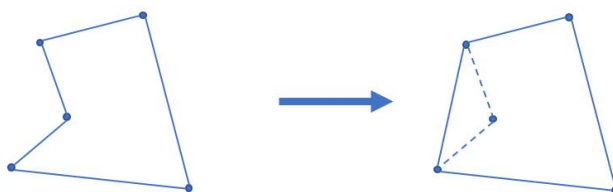
在平面上能包含所有给定点的最小凸多边形叫做凸包。

其定义为：

对于给定集合 X ，所有包含 X 的凸集的交集 S 被称为 X 的凸包 。

实际上可以理解为用一个橡皮筋包含住所有给定点的形态。

凸包用最小的周长围住了给定的所有点。如果一个凹多边形围住了所有的点，它的周长一定不是最小，如下图。根据三角不等式，凸多边形在周长上一定是最优的。



凸包的求法

我们这里不详细讲解与证明凸包算法的原理, 其本质就是极角坐标排序, 我们主要是告诉大家如何使用凸包算法的模板。

凸包面积

题目描述:

麦兜是个淘气的孩子。一天, 他在玩钢笔的时候把墨水洒在了白色的墙上。再过一会, 麦兜妈就要回来了, 麦兜为了不让妈妈知道这件事情, 就想用一个白色的凸多边形把墙上的墨点盖住。你能告诉麦兜最小需要面积多大的凸多边形才能把这些墨点盖住吗? 现在, 给出了这些墨点的坐标, 请帮助麦兜计算出覆盖这些墨点的最小凸多边形的面积。

输入描述:

多组测试数据。第一行是一个整数 T , 表明一共有 T 组测试数据。 每组测试数据的第一行是一个正整数 N ($0 < N \leq 105$), 表明了墨点的数量。接下来的 N 行每行包含了两个整数 X_i 和 Y_i ($0 \leq X_i, Y_i \leq 2000$), 表示每个墨点的坐标。每行的坐标间可能包含多个空格。

输出描述:

每行输出一组测试数据的结果, 只需输出最小凸多边形的面积。面积是个实数, 小数点后面保留一位即可, 不需要多余的空格。

输入样例#:

```
2
4
0 0
1 0
0 1
1 1
2
0 0
0 1
```

输出样例#:

```
1.0
```

0.0

题目来源:

DreamJudge 1113

解题分析: 使用求凸包面积的算法模板即可。

参考代码

```
1. #include <bits/stdc++.h>
2. #include <math.h>
3. using namespace std;
4. const int maxn=1005;
5. inline double sqr(double x){return x*x;}
6. struct point{
7.     double x,y;
8.     point(){}
9.     point(double _x,double _y):x(_x),y(_y){}
10.    //判断两点相同
11.    bool operator ==(const point &b) const{
12.        return x-b.x==0 && y-b.y==0;
13.    }
14.    point operator -(const point &b) const{
15.        return point(x-b.x,y-b.y);
16.    }
17.    //叉积
18.    int operator ^(const point &b) const{
19.        return x*b.y-y*b.x;
20.    }
21.    //点积
22.    int operator *(const point &b) const{
23.        return x*b.x+y*b.y;
24.    }
25.    //重载小于号 求最左下角的点
26.    bool operator <(const point &b) const{
27.        return x-b.x==0?y-b.y<0:x<b.x;
28.    }
29. };
30. struct line{
31.     point s,e;
32.     line(){}
}
```



```

33.     line(point _s,point _e):s(_s),e(_e){}
34. };
35. double dis(point a,point b){ //求两点之间的距离
36.     return sqrt(1.0*sqr(a.x-b.x)+1.0*sqr(a.y-b.y));
37. }
38. struct polygon{
39.     int n;
40.     point p[maxn];
41.     void add(point q){p[n++]=q;}
42.     struct cmp{
43.         point p;
44.         cmp(const point &p0):p(p0){}
45.         bool operator()(const point &aa,const point &bb){
46.             point a=aa,b=bb;
47.             int k=(a-p)^(b-p);
48.             if (k==0){
49.                 return dis(a,p)-dis(b,p)<0;
50.             }
51.             return k>0;
52.         }
53.     };
54.     //极角排序 先找到左下角的点
55.     //重载好 point 的 '<'
56.     void norm(){
57.         point mi=p[0];
58.         for (int i=1;i<n;i++) mi=min(mi,p[i]);
59.         sort(p,p+n,cmp(mi));
60.     }
61.     //得到凸包, 点编号为 0--n-1
62.     void Graham(polygon &convex){
63.         norm();
64.         int &top=convex.n;
65.         top=0;
66.         if (n==1){
67.             top=1; convex.p[0]=p[0]; return ;
68.         }
69.         if (n==2){
70.             top=2; convex.p[0]=p[0]; convex.p[1]=p[1];
71.             if (convex.p[0]==convex.p[1]) top--;
72.             return ;
73.         }
74.         convex.p[0]=p[0]; convex.p[1]=p[1]; top=2;
75.         for (int i=2;i<n;i++){

```

```

76.         while (top>1 && ((convex.p[top-1]-convex.p[top-2])^(p[i]-convex.p[top-2]))<=0)
            top--;
77.         convex.p[top++]=p[i];
78.     }
79.     if (convex.n==2 && (convex.p[0]==convex.p[1])) convex.n--;
80. }
81. double getarea(){//求凸包的面积
82.     double sum=0;
83.     for (int i=0;i<n;i++) sum+=(p[i]^(p[(i+1)%n]));
84.     return sum/2.0;
85. }
86. };
87. polygon C;
88. int main(){
89.     int t;
90.     cin >> t;
91.     while (t--) {
92.         int n,x,y; cin >> n; C.n=0;
93.         for (int i=0;i<n;i++){
94.             cin >> x >> y;
95.             C.add(point(x,y));
96.         }
97.         polygon ans; C.Graham(ans);
98.         double res = ans.getarea();
99.         printf("%.11f\n", res);
100.    }
101.    return 0;
102. }

```

总结：对于机试中的计算几何来说，记住上面的算法模板并知道是如何使用的即可。

简单的变形应用

例：一片森林里有很多树，每棵树对应二维平面上的一个坐标，现在小明想要在森林里放牛，所以小明用线缠绕在树上划出牛栏的边界，已知每头牛都需要 50 单位面积的活动空间，请问小明最多可以放多少头牛？

解析：很明显，将树抽象成二维平面上的点，然后画一个凸多边形就是最大的面积，其实就是

求凸包的面积, 将凸包面积/50 即为可以放牛的数量。

题目练习

DreamJudge 1160 球的半径和体积

DreamJudge 1183 Freckles

DreamJudge 1211 矩形相交

DreamJudge 1596 球形空间产生器

DreamJudge 1615 多边形的面积

DreamJudge 1620 放牧

DreamJudge 1621 玩具



2.2 进阶背包问题

在高分篇中，我们学习了简单背包问题，和 01 背包问题。

当时我们用的二维数组来进行递推的，仔细思考就可以发现其实我们没有必要开这么大的数组。因为我们所有的递推都是从上一行推到下一行的，也就是说，我们只需要开两行的数组即可完成这个过程。我们还可以进一步压缩空间，前半段和后半段也可以进行滚动。

滚动数组（上下滚动）

```

1. #include <stdio.h>
2. #include <string.h>
3.
4. int main() {
5.     int dp[2][1005] = {0}; //只需要 0 和 1 进行滚动
6.     int w[1005];
7.     int s, n;
8.     while (scanf("%d%d", &s, &n) != EOF) {
9.         for (int i = 1; i <= n; i++)
10.             scanf("%d", &w[i]);
11.         memset(dp, 0, sizeof(dp));
12.         dp[0][0] = 1;
13.         for (int i = 1; i <= n; i++) {
14.             int now = i & 1;
15.             for (int j = s; j >= 0; j--) { //将 i-1 变成 i^1
16.                 if (dp[now^1][j] == 1) dp[now][j] = 1;
17.                 if (j - w[i] >= 0 && dp[now^1][j-w[i]] == 1) dp[now][j] = 1;
18.             }
19.         }
20.         if (dp[n&1][s] == 1) printf("YES\n");
21.         else printf("NO\n");
22.     }
23.     return 0;
24. }
```

滚动数组（前后滚动）

```

1. #include <stdio.h>
2. #include <string.h>
3.
4. int main() {
```

```

5.     int dp[1005] = {0}; //前后滚动
6.     int w[1005];
7.     int s, n;
8.     while (scanf("%d%d", &s, &n) != EOF) {
9.         for (int i = 1; i <= n; i++)
10.            scanf("%d", &w[i]);
11.        memset(dp, 0, sizeof(dp));
12.        dp[0] = 1;
13.        for (int i = 1; i <= n; i++) {
14.            for (int j = s; j >= 0; j--) { //从后往前枚举
15.                if (j - w[i] >= 0 && dp[j-w[i]] == 1) dp[j] = 1;
16.            }
17.        }
18.        if (dp[s] == 1) printf("YES\n");
19.        else printf("NO\n");
20.    }
21.    return 0;
22. }

```

完全背包

有 N 种物品和一个容量为 V 的背包，每种物品都有无限件可用。第 i 种物品的费用是 $c[i]$ ，价值是 $w[i]$ 。求解将哪些物品装入背包可使这些物品的费用总和不超过背包容量，且价值总和最大。

$$F[i][j] = \begin{cases} F[i-1][j] & , \quad \text{不放第} i \text{种物品} \\ F[i][j - C[i]] + W[i], & j \geq C[i] \text{至少放一件第} i \text{种物品} \end{cases}$$

完全背包问题

题目描述:

有 n 种（每一种有无数个）重量和价值分别为 W_i, V_i 的物品，现从这些物品中挑选出总量不超过 W 的物品，求所有方案中价值总和的最大值。

输入描述#:

输入包含多组测试用例，每一例的开头为两位整数 n, W ($1 \leq n \leq 10000, 1 \leq W \leq 1000$)，接下来有 n 行，每一行有两位整数 W_i, V_i ($1 \leq W_i \leq 10000, 1 \leq V_i \leq 100$)

输出描述#:

输出为一行，即所有方案中价值总和的最大值。

输入样例#:

```
3 4
1 2
2 5
3 7
3 5
2 3
3 4
4 5
```

输出样例#:

```
10
7
```

题目来源:

DreamJudge 1569

题目解析：这个题是一个完全背包的算法模板题，直接套用完全背包的模板即可。

参考代码

```
1. #include<bits/stdc++.h>
2. using namespace std;
3.
4. int n,W,v[10005],w[10005],dp[1005];
5. int main(){
6.     while(cin>>n>>W){
7.         for(int i=0;i<n;i++){
8.             cin>>w[i]>>v[i];
9.             memset(dp,0,sizeof(dp));
10.            for(int i=0;i<n;i++) //种类
11.                for(int j=w[i];j<=W;j+=w[i]) //重量从小到大枚举
12.                    dp[j]=max(dp[j],dp[j-w[i]]+v[i]);
13.            cout<<dp[W]<<endl;
14.        }
```

```
15.     return 0;  
16. }
```

特别说明：多重背包的问题几乎没有学校机试会考，如果学有余力的同学可以尝试自己去学习一下。暴力的做法几乎人人都会，一般如果考到都需要优化。一种方法是类似二分快速幂的思想用二进制方式拆分求解，另一种方法是用单调队列来维护。



2.3 毛毛虫算法

毛毛虫算法（官方称为尺取法）通常是指对数组保存一对下标（起点、终点），然后根据实际情况交替推进两个端点直到得出答案的方法，这种操作很像是毛毛虫（尺取虫）爬行的方式故得名。

Subsequence

题目描述：

给出了 N 个正整数 ($10 < N < 100\ 000$) 的序列，每个正整数均小于或等于 10000，并给出了一个正整数 S ($S < 100\ 000\ 000$)。编写程序以查找序列中连续元素的子序列的最小长度，其总和大于或等于 S 。

输入描述：

第一行是测试用例的数量。对于每个测试用例，程序必须从第一行读取数字 N 和 S ，并以一个间隔分隔开。序列号在测试用例的第二行中给出，以间隔分隔。输入将以文件结尾结束。

输出描述：

对于每种情况，程序都必须将结果打印在输出文件的单独一行上。如果没有答案，则打印 0。

输入样例#：

```
2
10 15
5 1 3 5 10 7 4 9 2 8
5 11
1 2 3 4 5
```

输出样例#：

```
2
3
```

题目来源：

DreamJudge 1570

题目解析：这是一个典型的毛毛虫算法的应用实例，下面详细讲解一下算法过程。

我们设以 a_s 开始总和最初大于 S 时的连续子序列为 $a_s + \dots + a_{t-1}$ ，这时

$$a_{s+1} + \dots + a_{t-2} < a_s + \dots + a_{t-2} < S$$

所以从 a_{s+1} 开始总和最初超过 S 的连续子序列如果是 $a_{s+1} + \dots + a_{t'-1}$ 的话, 则必然有 $t \leq t'$ 。利用这一性质便可以设计出如下算法:

- (1) 以 $s = t = \text{sum} = 0$ 初始化。
- (2) 只要依然有 $\text{sum} < S$, 就不断将 sum 增加 a_t , 并将 t 增加 1。
- (3) 如果(2)中无法满足 $\text{sum} \geq S$ 则终止。否则的话, 更新 $\text{res} = \min(\text{res}, t-s)$ 。
- (4) 将 sum 减去 a_s , s 增加 1 然后回到(2)。

参考代码

```

1. #include<bits/stdc++.h>
2. using namespace std;
3.
4. int a[100005];
5. int main() {
6.     int T, n, s;
7.     scanf("%d", &T);
8.     while(T--) {
9.         scanf("%d%d", &n, &s);
10.        for(int i = 1; i <= n; i++)
11.            scanf("%d", &a[i]);
12.        int l = 1, r = 1, sum = 0, ans = 0x3f3f3f3f;
13.        while(1) {
14.            while(sum < s && r <= n) sum += a[r++];
15.            if(sum < s) break;
16.            ans = min(ans, r-l);
17.            sum -= a[l++];
18.        }
19.        if(ans == 0x3f3f3f3f) printf("0\n");
20.        else printf("%d\n", ans);
21.    }
22.    return 0;
23. }
```

题目练习

DreamJudge 1591 逛画展

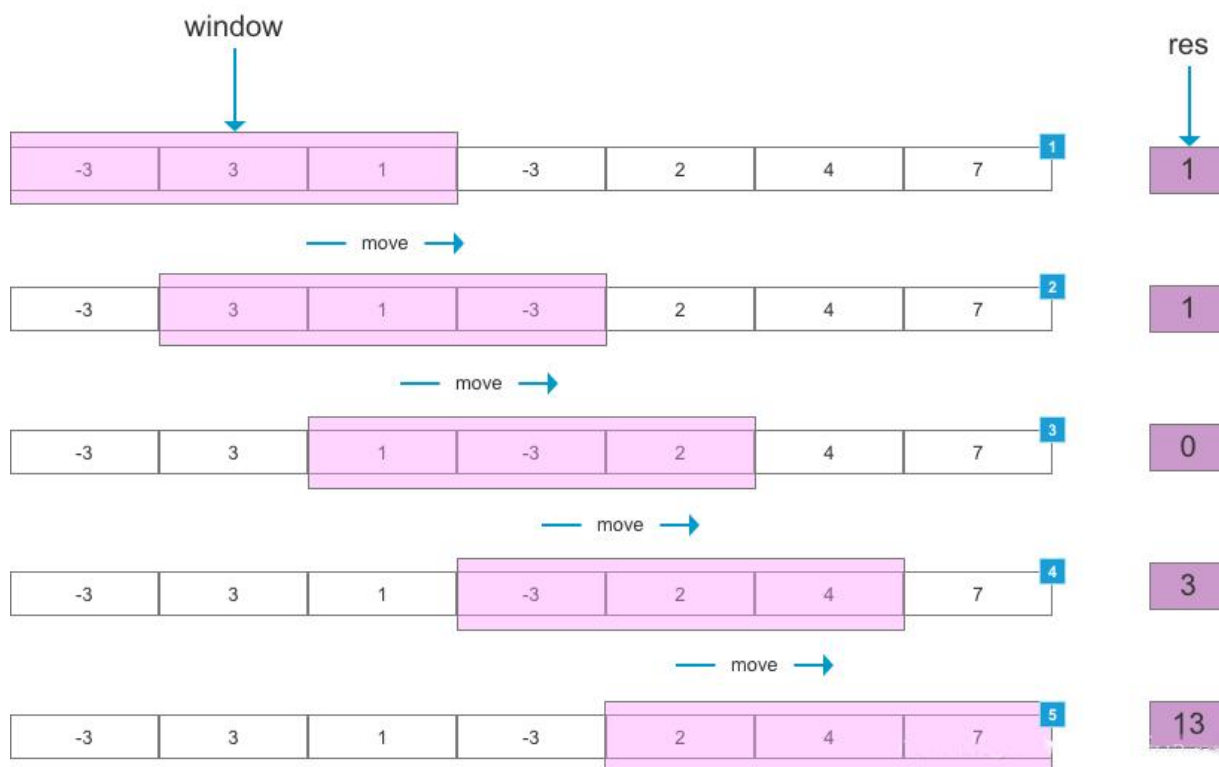
滑动窗口算法

毛毛虫算法有时候又叫做滑动窗口算法，我们可以换个方式来理解这类解题思想。

学过计算机网络的同学，都知道滑动窗口协议（Sliding Window Protocol），该协议是 TCP 协议 的一种应用，用于网络数据传输时的流量控制，以避免拥塞的发生。该协议允许发送方在停止并等待确认前发送多个数据分组。由于发送方不必每发一个分组就停下来等待确认。因此该协议可以加速数据的传输，提高网络吞吐量。

滑动窗口算法其实和这个是一样的，只是用的地方场景不一样，可以根据需要调整窗口的大小，有时也可以是固定窗口大小。

滑动窗口算法是在给定特定窗口大小的数组或字符串上执行要求的操作。该技术可以将一部分问题中的嵌套循环转变为一个单循环，因此它可以减少时间复杂度。简而言之，滑动窗口算法在一个特定大小的字符串或数组上进行操作，而不在整个字符串和数组上操作，这样就降低了问题的复杂度，从而也达到降低了循环的嵌套深度。其实这里就可以看出来滑动窗口主要应用在**数组**和**字符串**上。



可以用来解决一些查找满足一定条件的连续区间的性质（长度等）的问题。由于区间连续，因此当区间发生变化时，可以通过旧有的计算结果对搜索空间进行剪枝，这样便减少了重复计算，降低了时间复杂度。往往类似于“请找到满足 xx 的最 x 的区间（子串、子数组）的 xx ”这类问题都可以使用该方法进行解决。

需要注意的是，滑动窗口算法更多的是一种思想，而非某种数据结构的使用。

实战例子

例题：尽可能使字符串相等

给你两个长度相同的字符串， s 和 t 。

将 s 中的第 i 个字符变到 t 中的第 i 个字符需要 $|s[i] - t[i]|$ 的开销（开销可能为 0），也就是两个字符的 ASCII 码值的差的绝对值。

用于变更字符串的最大预算是 $maxCost$ 。在转化字符串时，总开销应当小于等于该预算，这也意味着字符串的转化可能是不完全的。

如果你可以将 s 的子字符串转化为它在 t 中对应的子字符串，则返回可以转化的最大长度。如果 s 中没有子字符串可以转化成 t 中对应的子字符串，则返回 0。

示例 1：

输入： $s = "abcd"$ ， $t = "bcdf"$ ， $cost = 3$

输出：3

解释： s 中的 $"abc"$ 可以变为 $"bcd"$ 。开销为 3，所以最大长度为 3。

示例 2：

输入： $s = "abcd"$ ， $t = "cdef"$ ， $cost = 3$

输出：1

解释： s 中的任一字符要想变成 t 中对应的字符，其开销都是 2。因此，最大长度为 1。

示例 3：

输入： $s = "abcd"$ ， $t = "acde"$ ， $cost = 0$

输出：1

解释：你无法作出任何改动，所以最大长度为 1。

2.4 博弈类问题

我们把机试中会考到的博弈问题分为了下面四类

- 1、简单博弈问题
- 2、巴什博弈
- 3、尼姆博弈
- 4、威佐夫博弈

博弈类的题目大多都是考察同学们推理和观察能力，大多数题目背景都是两个人做游戏，并且两个人都足够聪明，然后判断最终谁会取得胜利。

简单博弈

一眼能看出胜败关系的博弈或者存在必胜或必败的博弈。

简单博弈

题目描述:

有 $2*n$ 个硬币放在桌子上围成一个圆。现在甲和乙依次轮流拿硬币，每人每次最多拿 k 枚硬币，并且这 k 枚硬币必须相邻才能拿，每次最少要拿走 1 枚硬币，没有硬币拿的一方输掉比赛。

假设两个人都足够聪明，甲先拿，问甲能取得最后的胜利吗？

输入描述:

多组数据输入。

输入两个正整数 n 和 k 。 ($k \leq n$, $n \leq 10^9$)

输出描述:

如果甲一定能赢则输出“WIN”，如果甲一定会输则输出“LOSE”，如果不能确定胜负则输出“?”。

输入样例#:

1 1

输出样例#:

LOSE

题目来源:

DreamJudge 1632

题目解析: 很多同学刚看的时候觉得很难推出胜负关系, 那是没有找到题目的重点, 显然这个题的重点是圆, 圆具有从任何方向都对称的性质。使用对称原理, 不论甲拿几个, 乙都可以在对称的位置拿一样的数量, 这样不管甲怎么拿, 都是乙必胜。

参考代码

```
1. #include<stdio.h>
2.
3. int main() {
4.     int n, k;
5.     while (scanf("%d%d", &n, &k) != EOF) {
6.         printf("LOSE\n");
7.     }
8.     return 0;
9. }
```

noobdream.com

简单变种问题

两个人轮流往一张圆桌上放硬币（硬币须全部在桌面上），当一方没有位置可放的时候，另一方获胜。问是否有一种策略可以判断是先手获胜还是后手获胜？如果有，策略是什么？

提示: 也是利用圆的对称性

巴什博弈

两个顶尖聪明的人在玩游戏，有 n 个石子，每人可以随便拿 $1-m$ 个石子，不能拿的人为败者，问谁会胜利？

巴什博弈是博弈论问题中基础的问题，它是最简单的一种情形对应一种状态的博弈。

我们从最简单的情景开始分析

当石子有 $1-m$ 个时，毫无疑问，先手必胜

当石子有 $m+1$ 个时，先手无论拿几个，后手都可以拿干净，先手必败

当石子有 $m+2-2m$ 时，先手可以拿走几个，剩下 $m+1$ 个，先手必胜

我们不难发现，面临 $m+1$ 个石子的人一定失败。

这样的话两个人的最优策略一定是通过拿走石子，使得对方拿石子时还有 $m+1$ 个

我们考虑往一般情况推广

设当前的石子数为 $n=k*(m+1)+r$

先手会首先拿走 r 个，接下来假设后手拿走 x 个，先手会拿走 $m+1-x$ 个，这样博弈下去后手最终一定失败

设当前的石子数为 $n=k*(m+1)$

假设先手拿 x 个，后手一定会拿 $m+1-x$ 个，这样下去先手一定失败

Brave Game

题目描述:

十年前读大学的时候，中国每年都要从国外引进一些电影大片，其中有一部电影就叫《勇敢者的游戏》（英文名称：Zathura），一直到现在，我依然对于电影中的部分电脑特技印象深刻。今天，大家选择上机考试，就是一种勇敢（brave）的选择；这个短学期，我们讲的是博弈（game）专题；所以，大家现在玩的也是“勇敢者的游戏”，这也是我命名这个题目的原因。当然，除了“勇敢”，我还希望看到“诚信”，无论考试成绩如何，希望看到的都是一个真实

的结果，我也相信大家一定能做到的~

各位勇敢者要玩的第一个游戏是什么呢？很简单，它是这样定义的：

- 1、 本游戏是一个二人游戏；
- 2、 有一堆石子一共有 n 个；
- 3、 两人轮流进行；
- 4、 每走一步可以取走 $1 \cdots m$ 个石子；
- 5、 最先取光石子的一方为胜；

如果游戏的双方使用的都是最优策略，请输出哪个人能赢。

输入描述：

如果先走的人能赢，请输出“first”，否则请输出“second”，每个实例的输出占一行。

输出描述：

共 T 行，如果对于这组数据存在先手必胜策略则输出 Yes，否则输出 No，每个单词一行。

输入样例#：

2

23 2

4 3

输出样例#：

first

second

题目来源：

DreamJudge 1636

题目解析：我们可以发现，当 $n=0$ 的时候后手必胜(设其为 P 态), $n=1 \sim m$ 这几种状态由于先手可以一次全部取完导致先手必胜(设其为 N 态)。

接着当 $n=m+1$ 时，因为先手无论取走几个都会使后手一次全部取走导致后手必胜；

接着当 $n=m+2 \sim 2*m$ 时，因为先手可以一次就将石子取剩 $m+1$ 个，导致无论后手取几个，最后都会使先手一次全部取完，导致先手必胜。

综上所述, 不难发现一个人想要获胜就要把对方逼到当对方选择时棋子剩 $m+1$ 的倍数个, 由于先手的先发性, 只要 $n \neq (m+1)$ 的倍数, 先手就一定可以一次就把石子取到剩 $(m+1)$ 的倍数个, 不断逼使后手一直走必败路线。所以当 $n \%(m+1) = 0$ 时, 后手必胜, 否则先手必胜。

参考代码

```
1. #include<stdio.h>
2.
3. int main () {
4.     int t;
5.     scanf("%d", &t);
6.     while(t--) {
7.         int n, k;
8.         scanf("%d%d", &n, &k);
9.         if(n % (k + 1) == 0){
10.            printf("second\n");
11.        }
12.        else{
13.            printf("first\n");
14.        }
15.    }
16.    return 0;
17. }
```


尼姆博弈

有两个顶尖聪明的人在玩游戏，游戏规则是这样的：

有 n 堆石子，两个人可以从任意一堆石子中拿任意多个石子(不能不拿)，没法拿的人失败。问谁会胜利？

nim 游戏是巴什博弈的升级版，它不再是简单的一个状态，因此分析起来也棘手许多。

如果说巴什博弈仅仅博弈论的一个引子的话，nim 游戏就差不多算是真正的入门了。

定理

对于 nim 游戏，前辈们发现了一条重要的规律！

当 n 堆石子的数量异或和等于 0 时，先手必胜，否则先手必败

简单证明（可略过）

设 $\oplus$ 表示异或运算

nim 游戏的必败态我们是知道的，就是当前 n 堆石子的数量都为零

设 $a[i]$ 表示第 i 堆石子的数量，那么当前局面就是

$$0 \oplus 0 \oplus 0 \oplus \dots \oplus 0 = 0$$

对于先手来说，如果当前局面是

$$a_1 \oplus a_2 \oplus a_3 \oplus \dots \oplus a_n = k$$

那么一定存在某个 a_i ，它的二进制表示在最高位 k 上一定是 1

我们将 $a_i \oplus k$ ，这样就变成了

$$a_1 \oplus a_2 \oplus a_3 \oplus \dots \oplus a_n \oplus k = 0$$

此时先手必胜

对于先手来说，如果当前局面是

$$a_1 \oplus a_2 \oplus a_3 \oplus \dots \oplus a_n = 0$$

那么我们不可能将某一个 a_i 异或一个数字后使得

$$a_1 \oplus a_2 \oplus a_3 \oplus \dots \oplus a_n = 0$$

此时先手必败

nim 游戏

题目描述：

甲，乙两个人玩 Nim 取石子游戏。

Nim 游戏的规则是这样的：地上有 n 堆石子（每堆石子数量小于 10^4 ），每人每次可从任意一堆石子里取出任意多枚石子扔掉，可以取完，不能不取。每次只能从一堆里取。最后没石子可取的人就输了。假如甲是先手，且告诉你这 n 堆石子的数量，他想知道是否存在先手必胜的策略。

输入描述：

本题有多组测试数据。

第一行一个整数 T ($T \leq 10$)，表示有 T 组数据

接下来每两行是一组数据，第一行一个整数 n ，表示有 n 堆石子， $n \leq 10000$ 。

第二行有 n 个数，表示每一堆石子的数量。

输出描述：

共 T 行，如果对于这组数据存在先手必胜策略则输出 Yes，否则输出 No，每个单词一行。

输入样例#：

```
2
2
1 1
2
1 0
```

输出样例#：

```
No
Yes
```

题目来源：

DreamJudge 1635

题目解析：根据 NIM 游戏的定理当 n 堆石子的数量异或和等于 0 时，先手必胜，否则先手必败即可。

参考代码

```
1. #include<stdio.h>
```

```
2.  
3. int main(){  
4.     int t, n;  
5.     scanf("%d", &t);  
6.     while(t--){  
7.         scanf("%d", &n);  
8.         int ans = 0;  
9.         for(int i = 1; i <= n; ++i){  
10.            int shu;  
11.            scanf("%d", &shu);  
12.            ans ^= shu;  
13.        }  
14.        if(!ans) printf("No\n");  
15.        else printf("Yes\n");  
16.    }  
17.    return 0;  
18. }
```



威佐夫博弈

noobdream.com

有两堆石子，两个顶尖聪明的人在玩游戏，每次每个人可以从任意一堆石子中取任意多的石子或者从两堆石子中取同样多的石子，不能取得人输，分析谁会获得胜利？

威佐夫博弈不同于 Nim 游戏与巴什博奕，它的特殊之处在于不能将两堆石子分开分析。证明略。

定理

假设两堆石子为 (x, y) （其中 $x < y$ ）

那么先手必败，当且仅当

$$(y - x) * \frac{(\sqrt{5}+1)}{2} = x$$

其中的 $\frac{(\sqrt{5}+1)}{2}$ 实际就是 1.6181.618，黄金分割数！怎么样，博弈论是不是很神奇？

取石子游戏

题目描述：

有两堆石子，数量任意，可以不同。游戏开始由两个人轮流取石子。游戏规定，每次有两种不同的取法，一是可以在任意的一堆中取走任意多的石子；二是可以在两堆中同时取走相同数量的石子。最后把石子全部取完者为胜者。现在给出初始的两堆石子的数目，你先取，假设双方都采取最好的策略，问最后你是胜者还是败者。

输入描述：

多组测试数据。

一行输入两个数 a, b，表示石子的初始情况。

a, b <= 1 000 000 000

输出描述：

输出共一行。

第一行为一个数字 1、0 或-1，如果最后你是胜利者则为 1；若失败则为 0；若结果无法确定则为-1。

输入样例#：

8 4

输出样例#：

1

题目来源：

DreamJudge 1637

题目解析：利用威佐夫博弈定理即可解决。

参考代码

```
1. #include<stdio.h>
2. #include<math.h>
3. #include<stdlib.h>
4.
5. main() {
6.     int a, b;
7.     while (scanf("%d%d", &a, &b) != EOF) {
8.         int temp = abs(a - b);
9.         int ans = temp*(1.0+sqrt(5.0))/2.0;
10.        if (b < a) a = b;
11.        if(ans == a) printf("0\n");
12.        else printf("1\n");
13.    }
14.    return 0;
15. }
```



noobdream.com

题目练习

DreamJudge 1633 Euclid's Game

DreamJudge 1634 A interesting game